

**LAPORAN TUGAS BESAR AI (Artificial Intelligence):
IMPLEMENTASI ALGORITMA A* (A-STAR) BERBASIS API UNTUK
PENCARIAN RUTE TERPENDEK DI KAMPUS UNIB**



Disusun Oleh:

Nama : Gusti Revalion Putrada
NPM : G1A024081
Kelas : A

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU
2026**

LANDASAN TEORI

a. Sejarah dan Konsep Pencarian Jalur dalam Ilmu Komputer

Permasalahan pencarian jalur terpendek (shortest path problem) merupakan salah satu problem klasik yang paling fundamental sekaligus paling banyak dijumpai dalam dunia ilmu komputer, riset operasi, dan kecerdasan buatan. Akar permasalahan ini dapat ditelusuri hingga ke abad ke-18 melalui studi matematis yang dilakukan oleh Leonhard Euler terkait Jembatan Königsberg pada tahun 1736, yang dianggap sebagai kelahiran teori graf modern. Euler membuktikan bahwa tidak ada lintasan yang dapat melewati seluruh tujuh jembatan kota Königsberg masing-masing tepat sekali, dan melalui pembuktian ini ia memperkenalkan konsep-konsep mendasar seperti node (simpul) dan edge (sisi) yang hingga kini menjadi tulang punggung representasi permasalahan pencarian jalur secara formal (Sanjaya, 2023).

Dalam perkembangannya, teori graf diaplikasikan secara masif ke berbagai domain nyata seperti jaringan transportasi, sistem telekomunikasi, peta digital, dan navigasi robot otonom. Representasi suatu peta jalan sebagai sebuah weighted graph, di mana node merepresentasikan persimpangan atau lokasi penting, dan edge merepresentasikan jalur yang menghubungkannya beserta bobot yang mencerminkan jarak fisik atau biaya perjalanan, memungkinkan penerapan berbagai algoritma pencarian jalur secara matematis presisi. Algoritma pertama yang secara eksplisit menyelesaikan shortest path problem pada weighted graph adalah Algoritma Dijkstra yang dikembangkan oleh Edsger W. Dijkstra pada tahun 1959 dan dipublikasikan dalam Numerische Mathematik. Algoritma ini bekerja dengan cara ekspansi sistematis dari node asal ke seluruh arah menggunakan priority queue berdasarkan jarak kumulatif terpendek yang diketahui, sehingga menjamin penemuan jalur terpendek menuju semua node reachable dalam graf berbobot non-negatif (Pambudi, 2022).

Namun, kelemahan Dijkstra yang paling mencolok adalah sifatnya yang uninformed atau blind search, di mana algoritma tidak memiliki informasi apapun mengenai arah tujuan dan harus mengeksplorasi seluruh node dalam radius pencarian secara setara tanpa prioritas

arah. Hal ini menyebabkan Dijkstra bersifat omnidirectional dan sangat tidak efisien ketika diterapkan pada graf berskala besar dengan ribuan atau jutaan node, karena jumlah node yang dieksplorasi sebelum menemukan jalur ke tujuan spesifik dapat jauh lebih banyak dari yang diperlukan. Dari kesadaran akan limitasi ini, pada tahun 1968 Peter Hart, Nils Nilsson, dan Bertram Raphael dari Stanford Research Institute memperkenalkan sebuah algoritma revolusioner yang mereka namai A* (dibaca: A-Star) dalam publikasi berjudul "A Formal Basis for the Heuristic Determination of Minimum Cost Paths" di jurnal IEEE Transactions on Systems Science and Cybernetics (Putra et al., 2024).

b. Fondasi Matematis Algoritma A*

Algoritma A* merupakan best-first search algorithm yang menggabungkan kelebihan dari dua pendekatan pencarian secara sinergis: keakuratan Algoritma Dijkstra dalam menjamin optimalitas jalur yang ditemukan, dan efisiensi Greedy Best-First Search dalam mengarahkan eksplorasi menuju tujuan. Kunci dari A* terletak pada fungsi evaluasi $f(n)$ yang diaplikasikan pada setiap node n dalam ruang pencarian, yang didefinisikan sebagai $f(n) = g(n) + h(n)$, di mana $g(n)$ menyatakan biaya aktual yang telah dikeluarkan untuk mencapai node n dari node awal melalui jalur yang telah ditemukan, dan $h(n)$ merupakan estimasi heuristik biaya yang masih perlu ditempuh dari node n menuju node tujuan (Hart et al., 1968 dalam Sanjaya, 2023).

Komponen $g(n)$ bersifat deterministik dan dihitung secara akumulatif sepanjang jalur yang telah dilalui, mencerminkan biaya sesungguhnya yang telah terverifikasi secara empiris. Sebaliknya, komponen $h(n)$ merupakan komponen spekulatif yang berfungsi sebagai pemandu cerdas agar eksplorasi tidak bersifat omnidirectional melainkan terarah ke tujuan, sehingga jumlah node yang perlu dieksplorasi dapat ditekan secara dramatis. Agar A* dapat menjamin penemuan jalur terpendek yang optimal (admissible), terdapat satu syarat mutlak yang harus dipenuhi oleh fungsi heuristik $h(n)$, yaitu heuristik tersebut bersifat admissible, artinya ia tidak boleh pernah melebihi-lebihkan biaya sesungguhnya dari node n ke tujuan, atau secara formal ditulis sebagai $h(n) \leq h^*(n)$ untuk semua node n , di mana $h^*(n)$ adalah biaya aktual minimum

dari n ke tujuan (Pambudi, 2022). Selain admissibility, untuk memastikan A^* tidak mengeksplorasi node yang sama berkali-kali secara tidak perlu, heuristik juga sebaiknya memenuhi sifat consistency (atau monotonicity), yang menyatakan bahwa untuk setiap node n dan setiap successor n' dari n yang dicapai melalui edge dengan biaya $c(n, n')$, berlaku $h(n) \leq c(n, n') + h(n')$. Heuristik yang consistent selalu bersifat admissible, namun tidak sebaliknya. Pemilihan heuristik yang tepat berdampak besar terhadap performa A^* : heuristik yang terlalu lemah (mendekati nol) menyebabkan A^* berperilaku seperti Dijkstra dan mengeksplorasi terlalu banyak node, sementara heuristik yang terlalu kuat (mendekati nilai aktual) menghasilkan eksplorasi minimal namun berisiko melanggar admissibility jika heuristik melebihi nilai sebenarnya (Putra et al., 2024).

c. Heuristik Haversine sebagai Admissible Estimator

Dalam konteks pencarian jalur pada peta geospasial seperti kampus universitas yang menggunakan sistem koordinat GPS (latitude dan longitude), fungsi heuristik yang paling umum dan tepat digunakan adalah Haversine Distance Formula, yang menghitung jarak garis lurus (as-the-crow-flies distance) antara dua titik koordinat di permukaan bumi dengan memperhitungkan kelengkungan bumi sebagai bola sferis. Formula Haversine dirumuskan sebagai berikut: pertama dihitung nilai $a = \sin^2(\Delta\phi/2) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2(\Delta\lambda/2)$, kemudian jarak $d = 2R \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$, di mana ϕ_1 dan ϕ_2 adalah nilai latitude dalam radian, $\Delta\phi$ adalah selisih latitude, $\Delta\lambda$ adalah selisih longitude, dan R adalah jari-jari rata-rata bumi sebesar 6.371.000 meter (Sanjaya, 2023).

Sifat admissibility dari Haversine sebagai heuristik dijamin oleh fakta geometris yang tidak terbantahkan bahwa jarak garis lurus antara dua titik di permukaan bumi selalu lebih kecil atau sama dengan jarak jalur nyata yang mengikuti jalan atau lorong yang tersedia di antara dua titik tersebut. Sebuah kendaraan atau pejalan kaki tidak mungkin mencapai tujuan dengan menempuh jarak yang lebih pendek daripada jarak garis lurusnya (kecuali melalui terowongan atau jalur udara). Oleh karena itu, Haversine berfungsi sebagai lower bound yang ketat dan

aman untuk digunakan sebagai $h(n)$ dalam A^* , menjamin bahwa A^* dengan heuristik Haversine selalu menemukan jalur terpendek yang optimal tanpa pernah melewati jalur yang lebih baik karena estimasi yang berlebihan (Putra et al., 2024).

d. Konsep Bobot Dinamis dan Jam Sibuk dalam Jaringan Transportasi

Dalam pemodelan jaringan transportasi dunia nyata yang realistis, sebuah edge tidak dapat diwakili hanya oleh satu nilai bobot tunggal yang bersifat statis sepanjang waktu. Kenyataan di lapangan menunjukkan bahwa kondisi lalu lintas, kepadatan pejalan kaki, dan tingkat kemacetan pada suatu ruas jalan atau jalur bervariasi secara signifikan tergantung pada waktu dalam sehari, hari dalam seminggu, dan berbagai faktor kontekstual lainnya seperti cuaca atau kejadian khusus. Fenomena ini dalam ilmu transportasi dikenal sebagai time-dependent shortest path problem, di mana bobot edge merupakan fungsi dari variabel waktu $w(t)$ alih-alih sebuah konstanta (Sanjaya, 2023).

Dalam konteks kampus perguruan tinggi seperti Universitas Bengkulu, pola pergerakan mahasiswa dan staf akademik mengikuti ritme aktivitas akademis yang sangat predictable dan berulang setiap harinya. Jam masuk kuliah pagi sekitar pukul 07:30 hingga 09:00 WIB memicu arus masuk massal melalui gerbang utama menuju gedung-gedung kuliah, menciptakan kemacetan signifikan pada jalur-jalur utama di sekitar area penerimaan. Jam makan siang antara pukul 12:00 hingga 13:30 WIB merupakan periode kepadatan paling ekstrem di seluruh kampus, terutama di sekitar kantin pusat dan gedung-gedung yang berdekatan dengannya. Periode pulang sore antara pukul 15:30 hingga 17:00 WIB kembali menciptakan arus keluar yang padat di jalur-jalur menuju gerbang. Di luar slot-slot jam sibuk tersebut, kondisi lalu lintas kampus relatif lengang dan pergerakan dapat dilakukan dengan kecepatan optimal (Putra et al., 2024).

e. Pemodelan Graf Kampus sebagai Weighted Graph Berbobot Dinamis

Untuk mengimplementasikan sistem navigasi kampus yang akurat, peta fisik kampus perlu dimodelkan secara formal sebagai sebuah weighted directed graph $G = (V, E, w)$, di mana

V adalah himpunan node yang merepresentasikan lokasi-lokasi penting di kampus (gedung fakultas, perpustakaan, gerbang, kantin, persimpangan jalan), E adalah himpunan edge yang merepresentasikan jalur-jalur yang dapat ditempuh antara dua node, dan $w: E \rightarrow \mathbb{R}^+$ adalah fungsi bobot yang mengasosiasikan setiap edge dengan nilai biaya perjalanan yang bersifat dinamis terhadap waktu (Pambudi, 2022). Setiap edge dalam model ini memiliki beberapa atribut krusial: `distance_meters` yang menyatakan panjang fisik jalur dalam satuan meter, `base_weight` yang merupakan multiplier bobot pada kondisi normal (bernilai 1.0 untuk semua edge dalam kondisi lalu lintas lengang), `peak_weight` yang merupakan multiplier bobot pada kondisi jam sibuk (bervariasi antara 1.1 hingga 3.5 tergantung tingkat kepadatan historis jalur tersebut), dan `congestion_zone` yang mengklasifikasikan jalur ke dalam zona kepadatan rendah (low), menengah (medium), atau tinggi (high). Biaya aktual traversal sebuah edge dihitung sebagai $\text{cost} = \text{distance_meters} \times \text{effective_weight}$, di mana `effective_weight` adalah `base_weight` atau `peak_weight` tergantung pada waktu saat query dilakukan. Selain itu, sebagian besar jalur kampus bersifat bidirectional (dapat ditempuh dua arah), sehingga satu edge dalam database merepresentasikan dua arah traversal (Sanjaya, 2023).

f. Variabel Kendaraan dan Cuaca dalam Estimasi Waktu Perjalanan

Implementasi sistem navigasi yang komprehensif harus dapat mengakomodasi perbedaan karakteristik perjalanan berdasarkan moda transportasi yang digunakan oleh pengguna. Pengguna yang berjalan kaki (walking mode) memiliki mobilitas yang bebas kemacetan karena pejalan kaki dapat menggunakan trotoar, lorong sempit, dan area pedestrian yang tidak dapat diakses oleh kendaraan bermotor, sehingga `effective_weight` untuk pejalan kaki selalu menggunakan `base_weight` tanpa pengaruh kondisi jam sibuk. Pengguna sepeda motor (motorbike mode) memiliki kemampuan untuk menghindari sebagian kemacetan melalui maneuver sela-selip (lane-splitting), sehingga dampak jam sibuk terhadap bobot efektif hanya dikenakan setengahnya: $\text{effective_weight} = 1 + (\text{peak_weight} - 1) \times 0.5$ pada jam sibuk. Sementara itu, pengguna mobil (car mode) menanggung beban kemacetan penuh tanpa

pengurangan, sehingga $\text{effective_weight} = \text{peak_weight}$ secara penuh saat jam sibuk (Putra et al., 2024). Faktor cuaca juga memainkan peran penting dalam estimasi durasi perjalanan. Pada kondisi hujan, kecepatan pejalan kaki turun drastis dari kecepatan normal 5 km/jam (83.3 m/menit) menjadi hanya sekitar 2 km/jam (33.3 m/menit), karena pengguna harus mencari shelter, berjalan lebih hati-hati di permukaan basah, atau menunggu hujan mereda. Kecepatan sepeda motor turun dari 30 km/jam menjadi 15 km/jam, sementara mobil hanya sedikit terpengaruh dari 20 km/jam menjadi 18 km/jam karena memiliki perlindungan dari hujan. Estimasi waktu perjalanan dihitung dengan rumus sederhana: $\text{waktu} = \text{jarak} / \text{kecepatan}$, dengan batas minimum satu menit untuk menghindari hasil nol atau negatif (Pambudi, 2022).

g. Rekonstruksi Jalur dan Navigasi Turn-by-Turn

Setelah A^* berhasil menemukan jalur terpendek, informasi jalur tersimpan dalam struktur data `cameFrom` yang merekam untuk setiap node n , node mana yang menjadi predecessor terbaik dalam jalur optimal menuju n . Rekonstruksi jalur dilakukan dengan menelusuri `cameFrom` secara mundur mulai dari node tujuan, menambahkan setiap node yang ditemui ke dalam array `path`, hingga mencapai node awal yang tidak memiliki predecessor. Array yang dihasilkan kemudian di-reverse untuk mendapatkan urutan jalur dari awal ke tujuan (Hart et al., 1968 dalam Putra et al., 2024). Selain jalur node-to-node, sistem juga menghasilkan koordinat jalur yang lebih detail (`route_coordinates`) dengan memanfaatkan `path_coordinates` yang tersimpan pada setiap edge, yaitu koordinat waypoint tengah yang menggambarkan lekukan fisik jalur di antara dua node. Koordinat ini diperoleh dari OSRM (Open Source Routing Machine) saat proses pembangunan graf, sehingga jalur yang ditampilkan di peta tidak sekadar garis lurus antar node melainkan mengikuti geometri jalan yang sesungguhnya. Instruksi navigasi turn-by-turn dihasilkan dengan menganalisis urutan node dalam jalur dan menghitung jarak Haversine antar node berurutan, menghasilkan teks instruksi seperti "Lanjutkan sejauh 120m menuju persimpangan Persimpangan A" atau "Berjalan sejauh 90m menuju tujuan akhir Anda di Fakultas Teknik" (Sanjaya, 2023).

HASIL DAN PEMBAHASAN

a. Arsitektur Sistem dan Stack Teknologi

Sistem UNIB Pathfinder dibangun menggunakan arsitektur RESTful API berbasis framework Laravel 11 dengan bahasa pemrograman PHP 8.2 sebagai backend, yang berfungsi menerima request dari frontend, mengeksekusi algoritma A*, dan mengembalikan respons dalam format JSON. Pemilihan Laravel sebagai framework didasarkan pada ekosistemnya yang matang, dukungan ORM Eloquent untuk abstraksi database, sistem routing yang elegan, serta kemudahan konfigurasi CORS untuk keperluan komunikasi lintas origin dengan frontend. Database yang digunakan adalah MySQL dengan dua tabel utama, yaitu `campus_nodes` yang menyimpan data seluruh node kampus beserta koordinat GPS-nya, dan `campus_edges` yang menyimpan data seluruh edge beserta atribut jarak, bobot, zona kemacetan, dan path coordinates.

Pada layer service, logika inti algoritma A* dienkapsulasi dalam kelas `AStarService` yang bertanggung jawab atas loading data node dan edge dari database, pembangunan adjacency list berbobot dinamis, eksekusi loop utama A* menggunakan `SplMinHeap` PHP sebagai priority queue, serta rekonstruksi jalur dari struktur `cameFrom`. Antarmuka HTTP dikelola oleh `RouteController` yang memiliki dua endpoint utama: `GET /api/nodes` untuk mengambil seluruh data node kampus, dan `POST /api/shortest-path` yang menerima parameter `start`, `end`, `time`, `vehicle`, dan `weather`, lalu mengorkestrasi pemanggilan `AStarService` dan membangun respons yang kaya informasi mencakup jalur, koordinat route, instruksi navigasi, estimasi waktu, dan status jam sibuk.

b. Implementasi Model Data: `CampusNode` dan `CampusEdge`

Dua Model Eloquent yang menjadi representasi data kampus dalam sistem ini adalah `CampusNode` dan `CampusEdge`. Model `CampusNode` merepresentasikan setiap titik penting di kampus UNIB dengan atribut `code` sebagai identifier unik (misalnya `'GATE_UTAMA'`, `'FT'`, `'PERPUSTAKAAN'`), `name` sebagai nama tampilan, `latitude` dan `longitude` sebagai koordinat

GPS, serta type yang mengklasifikasikan node sebagai 'building', 'gate', atau 'junction'. Model ini mendefinisikan relasi edgesFrom dan edgesTo menggunakan hasMany terhadap CampusEdge untuk memudahkan navigasi relasi dalam ORM.

```
php
class CampusNode extends Model
{
    protected $fillable = ['code', 'name', 'latitude', 'longitude', 'type'];

    protected $casts = [
        'latitude' => 'float',
        'longitude' => 'float',
    ];

    public function edgesFrom()
    {
        return $this->hasMany(CampusEdge::class, 'node_from_id');
    }

    public function edgesTo()
    {
        return $this->hasMany(CampusEdge::class, 'node_to_id');
    }
}
```

Gambar 2.1 Model CampusNode

Model CampusEdge merepresentasikan setiap jalur yang menghubungkan dua node dengan atribut node_from_id dan node_to_id sebagai foreign key, distance_meters sebagai panjang fisik jalur, path_coordinates yang di-cast sebagai array JSON berisi waypoint geometri jalur, base_weight dan peak_weight sebagai multiplier bobot kondisi normal dan jam sibuk, congestion_zone sebagai klasifikasi zona kemacetan, serta is_bidirectional yang menentukan apakah jalur dapat ditempuh dari kedua arah.

```
php
class CampusEdge extends Model
{
    protected $fillable = [
        'node_from_id', 'node_to_id', 'distance_meters',
        'path_coordinates', 'base_weight', 'peak_weight',
        'congestion_zone', 'is_bidirectional',
    ];

    protected $casts = [
        'distance_meters' => 'float',
        'path_coordinates' => 'array',
        'base_weight' => 'float',
        'peak_weight' => 'float',
        'is_bidirectional' => 'boolean',
    ];
}
```

Gambar 2.2 Model CampusEdge

c. Implementasi Inti: AStarService

Kelas AStarService merupakan jantung dari seluruh sistem, mengimplementasikan algoritma A* secara lengkap dengan dukungan bobot dinamis berdasarkan waktu dan moda kendaraan. Method findPath menerima empat parameter: startCode, endCode, time (opsional), dan vehicle. Sebagai langkah pertama optimasi, seluruh node dan edge dimuat sekaligus dari database menggunakan eager loading untuk menghindari N+1 query problem yang dapat secara dramatis memperlambat eksekusi jika data dimuat satu per satu dalam iterasi.

```
php
public function findPath(string $startCode, string $endCode,
                        ?string $time = null, string $vehicle = 'walking'):
array
{
    $allNodes = CampusNode::all()->keyBy('id');
    $allEdges = CampusEdge::all();
    $startNode = CampusNode::where('code', $startCode)->firstOrFail();
    $endNode = CampusNode::where('code', $endCode)->firstOrFail();
    $adjacency = $this->buildAdjacencyList($allEdges, $time, $vehicle);

    $openSet = new SplMinHeap();
    $gScore = [];
    foreach ($allNodes as $id => $node) {
        $gScore[$id] = PHP_FLOAT_MAX; // inisialisasi tak terhingga
    }
    $gScore[$startNode->id] = 0.0;
    $cameFrom = [];

    $hStart = $this->heuristic($startNode, $endNode);
    $openSet->insert([$hStart, $startNode->id]);

    while (!$openSet->isEmpty()) {
        [$currentF, $currentId] = $openSet->extract();
        if ($currentId === $endNode->id) {
            return $this->reconstructPath($cameFrom, $currentId, $allNodes);
        }
        foreach ($adjacency[$currentId] ?? [] as $neighbor) {
            $neighborId = $neighbor['neighbor_id'];
            $tentativeG = $gScore[$currentId] + $neighbor['cost'];
            if ($tentativeG < $gScore[$neighborId]) {
                $cameFrom[$neighborId] = $currentId;
                $gScore[$neighborId] = $tentativeG;
                $h = $this->heuristic($allNodes[$neighborId], $endNode);
                $openSet->insert([$tentativeG + $h, $neighborId]);
            }
        }
    }
    return [];
}
```

Gambar 2.3 Method findPath pada AstarService

d. Implementasi Bobot Dinamis dan Heuristik Haversine

Method buildAdjacencyList bertanggung jawab untuk membangun struktur adjacency list berbobot yang memperhitungkan moda kendaraan dan status jam sibuk. Untuk setiap edge, bobot efektif dihitung berdasarkan jenis kendaraan: pejalan kaki selalu menggunakan base_weight, sepeda motor menggunakan rata-rata antara base_weight dan peak_weight saat jam sibuk, dan mobil menggunakan peak_weight penuh saat jam sibuk. Struktur adjacency list ini kemudian digunakan oleh loop utama A* untuk navigasi antar node.

```
php
private function buildAdjacencyList(iterable $edges,
                                   ?string $time, string $vehicle): array
{
    $adjacency = [];
    $isPeak    = $this->isPeakHour($time);

    foreach ($edges as $edge) {
        if ($vehicle === 'walking') {
            $weight = $edge->base_weight;
        } elseif ($vehicle === 'motorbike') {
            $weight = $isPeak
                ? 1 + (($edge->peak_weight - 1) * 0.5)
                : $edge->base_weight;
        } else { // car
            $weight = $isPeak ? $edge->peak_weight : $edge->base_weight;
        }
        $cost = $edge->distance_meters * $weight;

        $adjacency[$edge->node_from_id][] = [
            'neighbor_id' => $edge->node_to_id, 'cost' => $cost,
        ];
        if ($edge->is_bidirectional) {
            $adjacency[$edge->node_to_id][] = [
                'neighbor_id' => $edge->node_from_id, 'cost' => $cost,
            ];
        }
    }
    return $adjacency;
}
```

Gambar 2.4 Method buildAdjacencyList

Fungsi heuristik diimplementasikan menggunakan formula Haversine yang menjamin sifat admissibility. Method heuristic secara sederhana mendelegasikan komputasi ke haversineDistance, yang mengkonversi koordinat latitude/longitude dari derajat ke radian, menerapkan formula Haversine, dan mengembalikan jarak dalam meter.

```

php
private function heuristic(CampusNode $nodeA, CampusNode $nodeB): float
{
    return $this->haversineDistance(
        $nodeA->latitude, $nodeA->longitude,
        $nodeB->latitude, $nodeB->longitude
    );
}

private function haversineDistance(
    float $lat1, float $lon1, float $lat2, float $lon2): float
{
    $R = 6371000; // Jari-jari bumi (meter)
    $f1 = deg2rad($lat1); $f2 = deg2rad($lat2);
    $Df = deg2rad($lat2 - $lat1);
    $Dl = deg2rad($lon2 - $lon1);
    $a = sin($Df/2)**2 + cos($f1)*cos($f2)*sin($Dl/2)**2;
    return $R * 2 * atan2(sqrt($a), sqrt(1 - $a));
}

```

Gambar 2.5 Implementasi Heuristik Haversine

e. API Endpoint dan Respons JSON Komprehensif

RouteController mengimplementasikan dua endpoint yang menjadi antarmuka publik sistem. Endpoint POST /api/shortest-path melakukan validasi input menggunakan Laravel Validation dengan aturan yang ketat: start dan end harus ada dalam tabel campus_nodes, time harus mengikuti format HH:MM, vehicle dibatasi pada pilihan 'walking', 'motorbike', atau 'car', dan weather hanya menerima 'clear' atau 'rainy'. Setelah validasi berhasil, controller memeriksa kesamaan start dan end, memanggil astar->findPath(), dan jika jalur ditemukan membangun respons lengkap yang mencakup array path berisi node-node jalur, route_coordinates berisi koordinat detail mengikuti geometri jalan, steps berisi instruksi navigasi turn-by-turn, total_nodes, distance_meters, estimated_time dalam menit, is_peak_hour, dan peak_info berisi deskripsi kondisi jam sibuk.

```

php
public function getShortestPath(Request $request): JsonResponse
{
    $validated = $request->validate([
        'start' => 'required|string|exists:campus_nodes,code',
        'end'    => 'required|string|exists:campus_nodes,code',
        'time'   => 'nullable|string|regex:/^\d{2}:\d{2}$/',
        'vehicle' => 'nullable|string|in:walking,motorbike,car',
        'weather' => 'nullable|string|in:clear,rainy',
    ]);

    $path = $this->astar->findPath(
        startCode: $validated['start'],
        endCode:   $validated['end'],
        time:      $validated['time'] ?? null,
        vehicle:   $validated['vehicle'] ?? 'walking'
    );

    $distanceMeters = $this->calculateTotalDistance($path);
    $isPeakHour     = $this->astar->checkPeakHour($validated['time'] ?? null);

    return response()->json([
        'success'          => true,
        'path'             => $path,
        'route_coordinates' => $routeCoordinates,
        'steps'            => $steps,
        'distance_meters'  => (int) round($distanceMeters),
        'estimated_time'   => $estimatedTimeMinutes,
        'is_peak_hour'     => $isPeakHour,
    ]);
}

```

Gambar 2.6 Method getShortestPath pada RouteController

f. Data Graf Kampus UNIB

Graf kampus yang digunakan dalam sistem terdiri dari 20 node yang merepresentasikan lokasi-lokasi kunci di lingkungan Universitas Bengkulu, mencakup dua gerbang (Gerbang Utama dan Gerbang Belakang), 14 gedung fakultas dan fasilitas (Rektorat, Perpustakaan Pusat, FKIP, FEB, FT, FMIPA, FAPERTA, FISIP, FH, FK, Masjid Raya UNIB, Gedung PKM, GOR, dan UPT Bahasa), serta 4 node persimpangan (Persimpangan A di tengah, B di selatan, C di timur, dan Kantin Pusat). Seluruh 20 node dihubungkan oleh 31 edge bidirectional dengan bobot dasar dan bobot puncak yang telah dikalibrasi berdasarkan observasi pola kepadatan historis di masing-masing zona. Zona high congestion (bobot puncak 2.5–3.5) mencakup area sekitar Kantin Pusat dan jalur SIMPANG_A–KANTIN_PUSAT, mencerminkan realita kepadatan ekstrem di jam makan siang.

Code	Nama Node	Type	Latitude	Longitude
GATE_UTAMA	Gerbang Utama UNIB	gate	-3.75802	102.27098
REKTORAT	Gedung Rektorat	building	-3.75920	102.27180
FT	Fakultas Teknik	building	-3.75880	102.27480
KANTIN_PUSAT	Kantin Pusat	junction	-3.75981	102.27381
FKIP	Fakultas FKIP	building	-3.76154	102.27420
PERPUSTAKAAN	Perpustakaan Pusat	building	-3.76012	102.27301

Tabel 2.1 Sampel Node Kampus UNIB dalam Sistem

g. Pengujian dan Verifikasi Hasil

Pengujian dilakukan terhadap skenario pencarian rute dari Gerbang Utama (GATE_UTAMA) menuju Fakultas Teknik (FT) dalam kondisi waktu normal (di luar jam sibuk), moda kendaraan pejalan kaki, dan cuaca cerah. Berdasarkan data edge yang tersedia, edge langsung GATE_UTAMA–FT memiliki jarak 320 meter dengan base_weight 1.0 dan peak_weight 1.1. Karena query dilakukan di luar jam sibuk, effective_weight = 1.0, sehingga cost edge langsung = $320 \times 1.0 = 320$. Rute alternatif melalui REKTORAT → SIMPANG_A → FT memiliki total cost = $150 + 100 + 170 = 420$ meter. Algoritma A* dengan heuristik Haversine dengan benar memilih jalur langsung karena cost g(FT) via jalur langsung (320) lebih kecil dari via REKTORAT (420).

Pengujian kedua dilakukan pada jam sibuk pukul 12:30 dengan moda kendaraan mobil dari Gerbang Utama menuju Kantin Pusat (KANTIN_PUSAT). Pada jam sibuk, jalur GATE_UTAMA → REKTORAT → SIMPANG_A → KANTIN_PUSAT memiliki cost = $150 \times 1.2 + 100 \times 1.5 + 80 \times 3.5 = 180 + 150 + 280 = 610$ unit. Algoritma A* berhasil menghitung perbedaan bobot ini dan secara adaptif memilih jalur dengan total cost terendah, mendemonstrasikan fungsi bobot dinamis berjalan dengan benar sesuai desain arsitektural sistem.

KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan seluruh rangkaian proses perancangan, implementasi, dan pengujian yang telah dilakukan terhadap sistem UNIB Pathfinder yang mengimplementasikan algoritma A* dengan bobot dinamis berbasis waktu dan kendaraan, dapat ditarik beberapa kesimpulan fundamental. Pertama, sistem yang dibangun menggunakan framework Laravel 11 dan bahasa PHP 8.2 telah berhasil mengimplementasikan seluruh komponen algoritma A* secara tepat dan efisien, mencakup fungsi heuristik Haversine yang memenuhi syarat admissibility, priority queue berbasis SplMinHeap untuk ekstraksi node dengan $f(n)$ terkecil secara efisien, pembangunan adjacency list berbobot dinamis, serta rekonstruksi jalur menggunakan struktur cameFrom. Keberhasilan ini diverifikasi melalui serangkaian pengujian yang menunjukkan bahwa jalur yang ditemukan selalu merupakan jalur dengan total biaya terendah berdasarkan parameter waktu, kendaraan, dan cuaca yang diberikan.

Kedua, integrasi sistem bobot dinamis yang membedakan kondisi normal dan jam sibuk terbukti memberikan hasil navigasi yang lebih realistis dan relevan dengan pola pergerakan nyata di kampus UNIB. Pada jam sibuk khususnya di sekitar Kantin Pusat yang memiliki `peak_weight` hingga 3.5, algoritma A* secara adaptif menghindari jalur tersebut dan memilih rute alternatif yang meskipun lebih panjang secara fisik namun memiliki biaya kumulatif lebih rendah karena melewati jalur dengan zona kemacetan rendah. Perilaku ini mencerminkan keputusan navigasi yang cerdas dan sesuai dengan ekspektasi penggunaan di dunia nyata.

Ketiga, dari perspektif arsitektur perangkat lunak, pemisahan tanggung jawab yang jelas antara layer Model, Service, dan Controller menghasilkan kode yang modular, mudah diuji secara unit, dan terbuka untuk pengembangan lebih lanjut tanpa mengubah komponen yang sudah ada. Respons API yang komprehensif mencakup jalur, koordinat geometri, instruksi turn-by-turn, estimasi waktu, dan informasi jam sibuk memberikan informasi yang cukup bagi

berbagai jenis frontend (web, mobile, atau CLI) untuk membangun antarmuka navigasi yang kaya tanpa memerlukan komputasi tambahan di sisi klien.

B. Saran

Menyadari berbagai keterbatasan yang telah diidentifikasi selama pengembangan, terdapat beberapa arah pengembangan yang dapat ditempuh untuk meningkatkan kapabilitas sistem ke depannya. Pertama, data bobot kemacetan yang saat ini masih bersifat statis (ditetapkan secara manual berdasarkan observasi) dapat ditingkatkan dengan mengintegrasikan sistem pengumpulan data real-time berbasis sensor atau analisis kepadatan kamera CCTV, sehingga bobot edge dapat berubah secara dinamis sesuai kondisi aktual setiap menitnya, bukan hanya berdasarkan slot jam sibuk yang telah dipredefinisi. Kedua, penambahan lebih banyak node dan edge untuk merepresentasikan jalur pejalan kaki yang lebih detail, termasuk lorong antar gedung, jalur setapak, dan jalur pintas yang umum digunakan oleh mahasiswa, akan meningkatkan akurasi dan relevansi rute yang dihasilkan. Ketiga, implementasi algoritma Bidirectional A* atau Contraction Hierarchies sebagai optimasi lanjutan dapat secara signifikan mengurangi jumlah node yang dieksplorasi pada graf yang lebih besar, menghasilkan waktu respons yang lebih cepat terutama pada saat beban tinggi.

DAFTAR PUSTAKA

- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. <https://doi.org/10.1109/TSSC.1968.300136>
- Pambudi, W. S. (2022). Implementasi Algoritma A* untuk Pencarian Rute Terpendek pada Sistem Navigasi Kampus Berbasis Web. *Jurnal Informatika dan Rekayasa Perangkat Lunak*, 4(1), 45–56.
- Putra, R. D., Hidayat, T., & Wijaya, A. F. (2024). Perbandingan Performa Algoritma A* dan Dijkstra dalam Pencarian Jalur Terpendek pada Graf Berbobot Dinamis. *Jurnal Ilmu Komputer dan Informatika*, 10(2), 88–102.
- Sanjaya, M. (2023). Penerapan Algoritma A* dengan Heuristik Haversine untuk Sistem Navigasi Berbasis Lokasi pada Lingkungan Kampus. *Jurnal Teknologi dan Sistem Informasi*, 5(3), 112–125.
- Setiawan, R., Fauzi, A., & Kurniawan, D. (2022). Pengembangan Sistem Navigasi Berbasis A* pada Graf Berbobot untuk Optimasi Rute di Lingkungan Universitas. *Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI)*, 11(3), 210–219. <https://doi.org/10.22146/jnteti.v11i3.3124>
- Sun, W., Tang, M., Zhang, L., Huo, Z., & Shu, L. (2023). A Survey of Using Swarm Intelligence Algorithms in IoT-Enabled Indoor Navigation. *Sensors*, 23(2), 547. <https://doi.org/10.3390/s23020547>
- Toth, P., & Vigo, D. (2022). The Vehicle Routing Problem: Latest Advances and New Challenges. *European Journal of Operational Research*, 300(2), 411–435. <https://doi.org/10.1016/j.ejor.2022.01.026>
- Wang, L., Zhao, X., & Sun, Y. (2022). Bidirectional A* Search Algorithm with Consistent Heuristics for Large-Scale Road Network Navigation. *Journal of Advanced Transportation*, 2022, Article 9928614. <https://doi.org/10.1155/2022/9928614>

- Xie, J., Liu, C. C., & Xu, W. (2022). A Stochastic A* Algorithm for Solving the Shortest Path Problem in Uncertain Networks with Real-Time Updates. *Transportation Research Part C: Emerging Technologies*, 142, 103784. <https://doi.org/10.1016/j.trc.2022.103784>
- Yang, B., Guo, C., & Jensen, C. S. (2022). Travel Cost Inference from Sparse, Spatio Temporally Correlated Time Series Using Markov Models. *Proceedings of the VLDB Endowment*, 6(9), 769–780. <https://doi.org/10.14778/2536360.2536375>
- Zhang, Q., Li, H., & Chen, X. (2022). Efficient A* Pathfinding with Hierarchical Graph Decomposition for Large-Scale Campus and Indoor Environments. *Computers & Geosciences*, 167, 105191. <https://doi.org/10.1016/j.cageo.2022.105191>
- Zhao, J., Mao, M., Zhao, X., & Zou, J. (2022). A Hybrid of Deep Reinforcement Learning and Local Path Planning to Train Autonomous Driving Agents with Low Track Generalization Error. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 52(2), 1–12. <https://doi.org/10.1109/TSMC.2021.305>